

PROJEKTBERICHT IM MODUL INTERNET OF THINGS UND CYBER-PHYSICAL
SYSTEMS

Smart Parking

**Konzeption und Umsetzung der dezentralen
Kennzeichenerkennung und Schrankensteuerung mittels Edge-KI
auf dem Raspberry Pi 5**

Studiengang: Master Informatik
Verfasser: Maximilian Strauß
Dozent/in: Prof. Dr. Jube
Abgabe: 19. Juli 2026

Ostbayerische Technische Hochschule Regensburg

Inhaltsverzeichnis

1	Spezifikation	1
1.1	Einführung und Motivation	1
1.2	Zielgruppe und Systemanforderungen	1
2	Analyse & Entwurf	1
2.1	Systemarchitektur und physisches Setup	1
2.2	Software-Architektur und KI-Pipeline	2
2.2.1	Funktionsweise der KI-Komponenten	2
2.2.2	CPU-spezifische PyTorch-Installation	2
2.2.3	NCNN-Format, Quantisierung und Multi-Threading	2
2.3	Entwurfsentscheidungen und Trade-offs	3
2.3.1	Wahl der OCR-Engine: EasyOCR vs. Tesseract	3
2.3.2	Verzicht auf ein spezielles Kennzeichen-Modell	3
2.3.3	Netzwerkinfrastruktur: WLAN vs. LAN	3
3	Implementierung & Tests	4
3.1	Konzept der LPR-Engine	4
3.2	Validierung und Stabilisierung	4
3.3	Peripherie-Steuerung	5
3.3.1	OLED-Display-Ansteuerung	5
3.3.2	Schrankensteuerung	5
3.4	Testverfahren	5
4	Evaluierung	5
4.1	Erkennungsgeschwindigkeit und Latenz	5
4.2	Arbeitsspeicher-Erweiterung	6
5	Reflektion	6
5.1	Gewonnene Erkenntnisse	6
5.2	Schwachstellen des physischen Prototyps	6
5.3	Zukünftige Verbesserungen	6

1 Spezifikation

1.1 Einführung und Motivation

In Cyber-Physical Systems und Internet of Things nimmt die intelligente Verkehrs- und Gebäudeinfrastruktur eine zentrale Rolle ein. Parkhäuser haben meistens ineffiziente Prozesse beim Ein- und Ausfahrtsprozess, die durch manuelle Ticket-Systeme verursacht werden.

Das Projekt „Smart Parking“ versucht durch ein automatisiertes Parkhaus-Modell dieses Problem zu beheben. Das System verfügt über eine automatische Nummernschilderkennung (License Plate Recognition, LPR) kombiniert mit einer Peripheriesteuerung (Schrannenmotoren, Status-Displays). Ziel ist es, die Zufahrtskontrolle an einem Edge-Computer durchzuführen.

1.2 Zielgruppe und Systemanforderungen

Die Zielgruppe für dieses System sind Parkhausbetreiber. Sie sollen in der Lage sein, die Ein- und Ausfahrt von Fahrzeugen effizient zu steuern, ohne dass manuelle Eingriffe erforderlich sind. Folgende Spezifikationen wurden für das System definiert:

- Dezentraler Edge-Ansatz: Die Bildverarbeitung und Kennzeichenerkennung muss direkt an den jeweiligen Stationen (Einfahrts- und Ausfahrtsschranke) stattfinden, um die Cloud- oder Serverinfrastruktur zu entlasten.
- Echtzeitnahe Erkennung: Die Erkennung des Kennzeichens und die Schrankenöffnung müssen schnell erfolgen, um Staus an den Schranken zu vermeiden.
- Hardware-Restriktionen: Jede Station ist aus Kostengründen mit einem kostengünstigen Einplatinencomputer (Raspberry Pi, ohne dedizierte GPU) ausgestattet.
- Peripheriesteuerung: Jede der beiden Stationen steuert lokal eine Kamera zur Zustandserfassung, ein kleines OLED-Display zur Fahrerkommunikation und einen Servomotor für die Schrankenbewegung.
- IoT-Konnektivität & Dashboard: Die erfassten Kennzeichen müssen über ein standardisiertes Protokoll (MQTT) an das Backend gemeldet werden. Ein Flask-basierter Videostream stellt das Kamerabild für die Anzeige im Web-Frontend zur Verfügung, damit Betreiber die Schranken überwachen können. Die Freigabe der Schranke erfolgt nach Prüfung des Zahlungs- oder Berechtigungsstatus im Backend.

2 Analyse & Entwurf

2.1 Systemarchitektur und physisches Setup

Das Gesamtsystem des intelligenten Parkhauses besteht aus zwei Computern (Einfahrts-Pi und Ausfahrts-Pi) sowie einem zentralen Gateway, auf dem der MQTT-Broker (Mosquitto), die Datenbank und die Logik des Backends ausgeführt werden. Jeder der beiden Raspberry Pis steuert folgende Komponenten an:

- Kamera: Eine Raspberry Pi Kamera, die über die native `PiCamera2`-API angesprochen wird.
- OLED-Display: Ein kleines I2C OLED-Display, das an einen Wemos D1 Mini (ESP8266) angeschlossen ist. Der D1 Mini wird über eine serielle USB-Schnittstelle vom Raspberry Pi gesteuert.

- Schrankenmotor: Ein 5V-Servomotor, der direkt über die GPIO-Pins des Raspberry Pi mit PWM-Signalen versorgt wird.

2.2 Software-Architektur und KI-Pipeline

Für die Erkennung der Kennzeichen wurde eine zweistufige Pipeline entworfen:

1. Lokalisierung (Object Detection): Erkennung des Kennzeichenbereichs (Region of Interest, ROI) im Videobild. Hierfür wird das Objekterkennungsmodell **YOLOv8n** (nano) eingesetzt. Das Nano-Modell ist die kleinste Variante der YOLOv8-Familie und wurde aufgrund der Hardware-Limitierungen des Raspberry Pi 5 gewählt.
2. Texterkennung (OCR): Extraktion der alphanumerischen Zeichen aus der erkannten ROI. Hierfür wird die OCR-Bibliothek **EasyOCR** verwendet.

2.2.1 Funktionsweise der KI-Komponenten

- YOLO (You Only Look Once): YOLO ist ein Deep-Learning-Algorithmus zur Objekterkennung. Im Gegensatz zu klassischen Erkennungsmethoden, die Bildbereiche schrittweise abtasten, betrachtet YOLO das gesamte Bild in einem einzigen Vorwärtspass eines neuronalen Netzes. Das Bild wird in ein Raster unterteilt, und das Netz sagt für jede Zelle Bounding-Boxes (Begrenzungsrahmen) und Klassenwahrscheinlichkeiten voraus. Im Parkhaus identifiziert YOLOv8n die Position des Nummernschilds im Kamerabild, liefert die Koordinaten für den Bildausschnitt und gibt einen Konfidenzwert zurück [1].
- OCR (Optical Character Recognition): Die Zeichenerkennung wandelt Bildinhalte eines Textbereichs in lesbare Zeichen um. EasyOCR kombiniert dafür drei neuronale Komponenten. Ein Convolutional Neural Network zur Merkmalsextraktion aus der ROI, ein Recurrent Neural Network zur Sequenzmodellierung (Erkennung von Zeichenzusammenhängen von links nach rechts) und einen CTC-Decoder (Connectionist Temporal Classification), der die berechneten Wahrscheinlichkeiten Buchstaben und Ziffern zuordnet [2].

2.2.2 CPU-spezifische PyTorch-Installation

Da der Raspberry Pi 5 über keine NVIDIA-GPU verfügt und CUDA-basierte Beschleunigung somit nicht möglich ist, wird bei der Einrichtung der Python-Umgebung die CPU-only-Variante von PyTorch installiert. Dies verhinderte das Herunterladen und Installieren von mehreren Gigabyte an CUDA-Laufzeitbibliotheken. Dadurch wird der Installationsprozess beschleunigt und der begrenzte Speicherplatz der MicroSD-Karte auf dem Raspberry Pi geschont.

2.2.3 NCNN-Format, Quantisierung und Multi-Threading

Um die Hardware-Limitierungen des Raspberry Pi 5 (1 GB RAM, CPU-Inferenz) zu bewältigen, wird folgendermaßen optimiert:

- NCNN-Format und Quantisierung: Das YOLOv8n-Modell wird vom Standard-PyTorch-Format (**.pt**) in das NCNN-Format konvertiert [3]. Bei der Konvertierung wird ein quantisiertes Modell verwendet. Die Quantisierung reduziert die Genauigkeit der Gewichte (z. B. von FP32 auf INT8/FP16), was den Speicherbedarf senkt und die Inferenzgeschwindigkeit auf CPUs erhöht, während die Ergebnisse auf ungefähr gleichem Level bleiben.

- **Multi-Threading:** Die Software wird in drei asynchrone Threads unterteilt, um ein Blockieren des Kamera-Feeds zu verhindern. Der `VideoCaptureThread` liest kontinuierlich Rohbilder ein, der `ProcessingThread` führt die KI-Pipeline in Intervallen aus, und der `Main/FlaskThread` zeichnet Bounding-Boxes und stellt den Videostream für das Frontend-Dashboard bereit.

2.3 Entwurfsentscheidungen und Trade-offs

2.3.1 Wahl der OCR-Engine: EasyOCR vs. Tesseract

Insgesamt werden zwei verschiedene OCR-Engines getestet:

- **Tesseract OCR:** Das schnellste Modell in Bezug auf die Verarbeitungsgeschwindigkeit ist Tesseract OCR. Die Inferenzzeit liegt unter 1 Sekunde auf der Pi-CPU. Allerdings ist die Erkennungsrate bei den Miniatur-Kennzeichen der Spielzeugautos unzuverlässig.
- **EasyOCR:** EasyOCR benötigt durch die Nutzung komplexer neuronaler Netze mehr Rechenleistung, was im Endeffekt zu der hohen Latenz des Gesamtsystems beiträgt. Im Gegenzug ist es jedoch deutlich robuster gegenüber den physikalischen Gegebenheiten unseres Prototyps, wie Neigungen, Schattenwurf und ungleichmäßige Schriftstärken, und konnte die Kennzeichen im Gegensatz zu Tesseract deutlich besser beziehungsweise überhaupt identifizieren.

Daher wurde die Entscheidung getroffen, EasyOCR als OCR-Engine zu verwenden, um eine höhere Erkennungsgenauigkeit zu gewährleisten, auch wenn dies auf Kosten der Geschwindigkeit geht [4].

2.3.2 Verzicht auf ein spezielles Kennzeichen-Modell

Es stellt sich die Frage, warum nicht direkt ein auf Kennzeichen vortrainiertes Objekterkennungsmodell, wie z. B. spezielle `ModernLPR_system` [5], verwendet wird. Diese Modelle sind auf echte Fahrzeuge mit realen Proportionen und realen Einbauwinkeln trainiert. Da in diesem Projekt ein Miniatur-Parkhaus mit Spielzeugautos simuliert wird, ist so ein Modell nicht aussagekräftig. Ein speziell auf echte Kennzeichen trainiertes Modell hätte in dieser Miniaturumgebung keine oder nur sehr schlechte Ergebnisse geliefert. Daher ist die Entscheidung auf das generische YOLOv8n-Modell in Kombination mit EasyOCR gefallen.

2.3.3 Netzwerkinfrastruktur: WLAN vs. LAN

In der ersten Entwurfsphase war geplant, die gesamte Kommunikation über das lokale WLAN des Backend-Servers abzuwickeln, allerdings führte dies zu folgenden Problemen:

- **Hohe Latenz:** Die kontinuierliche Übertragung der Videostreams für das Dashboard über das geteilte WLAN führte zu einer Überlastung und der Videostream wurde verzögert.
- **Broker-Überlastung:** Da der MQTT-Broker auf derselben Hardware lief, die auch den WLAN-Verkehr routen musste, kam es zu mehr Verbindungsproblemen und generellen Störungen bei der Entwicklung.

Das Netzwerk wurde auf eine LAN-Verkabelung umgestellt. Über einen Switch und einen Router wurden die IPs der Geräte verwaltet. Der Livestream läuft stabil und flüssig auf dem Admin-Dashboard auf, während die Internetausfälle und Latenzen auf ein Minimum reduziert wurden.

3 Implementierung & Tests

3.1 Konzept der LPR-Engine

Die LPR-Engine wurde als asynchroner Python-Dienst implementiert. Der Algorithmus durchläuft folgende Schritte:

- **Farb-Fix:** Der Zugriff auf die Kamera erfolgt über die `Picamera2`-Schnittstelle im `RGB888`-Format bei einer Auflösung von 1280×720 Pixeln. Da OpenCV intern standardmäßig mit der `BGR`-Farbkanalreihenfolge arbeitet, wird eine spezielle Konvertierung verwendet, um die Farbkanäle korrekt zu interpretieren (`cv2.COLOR_RGB2BGR`).
- **Durchsatz-Optimierung:** Zur Entlastung der CPU wird Frame-Dropping vorgenommen. Nur jeder fünfte erfasste Frame wird an den Inferenz-Thread weitergegeben (`PROCESS_EVERY_N_FRAMES = 5`), während die restlichen Frames verworfen werden.
- **ROI-Vorbereitung:** Sobald das quantisierte YOLOv8n-Modell eine Kennzeichen-Region mit einer Konfidenz von > 0.5 detektiert, werden die Koordinaten auf die Originalgröße zurückgerechnet. Der ROI wird in Graustufen konvertiert und auf eine Höhe von 120 Pixeln normiert. Eine anschließende Binarisierung nach dem Otsu-Verfahren (`cv2.THRESH_BINARY + cv2.THRESH_OTSU`) trennt die Schriftzeichen vom Hintergrund und verringert die Rechenlast für die nachfolgende OCR-Pipeline.

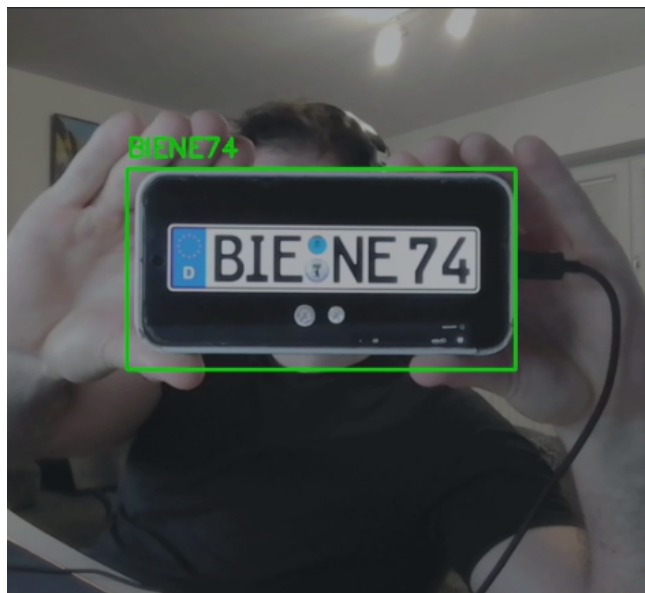


Abbildung 1: Visualisierung der Bounding Box um das Kennzeichen und des erkannten Texts (links oben) im Kamerabild vor dem Publizieren via MQTT

3.2 Validierung und Stabilisierung

Um Fehlöffnungen der Schranke durch Falscherkennungen zu vermeiden, wird ein dreistufiges Validierungskonzept umgesetzt:

1. **Syntaktische Validierung:** Kennzeichen müssen dem Regex-Muster deutscher Nummernschilder entsprechen.

2. Sliding Window Stabilisierung: Ein Kennzeichen wird in einem Puffer gesammelt. Es muss innerhalb eines definierten Zeitfensters (`STABILIZATION_WINDOW`) mindestens zweimal (`STABILIZATION_MIN_SAMPLES`) identisch erkannt werden, bevor es als stabil gilt und per MQTT veröffentlicht wird. Ein zeitlicher Debounce-Filter verhindert ein mehrfaches Triggern für dasselbe Fahrzeug.
3. Heuristische Fehlerkorrektur: Da die EasyOCR-Engine die kreisförmigen Zulassungs- und TÜV-Stempel auf deutschen Nummernschildern häufig fälschlicherweise als einen Buchstaben interpretiert, detektiert der Algorithmus solche Fehlpaarungen. Erkennt das System ein ungültiges Muster, das durch das Entfernen eines „S“ zwischen Buchstaben- und Ziffernblöcken valide wird, bereinigt ein RegEx-Filter das Textsegment vor der MQTT-Übergabe.

3.3 Peripherie-Steuerung

3.3.1 OLED-Display-Ansteuerung

Das OLED-Display zeigt im Ruhezustand den Text "BEREIT" an. Wird über den Node-RED-Flow ein neuer Status vom Backend empfangen, ruft Node-RED ein Python-Skript auf. Dieses stellt eine USB-Verbindung zum Wemos D1 Mini her und überträgt den anzuzeigenden Text (z. B. "Willkommen"). Nach Ablauf einer vordefinierten Anzeigedauer wird das OLED-Display automatisch wieder in den Bereit-Modus versetzt.

3.3.2 Schrankensteuerung

Die Steuerung des Servomotors erfolgt über ein weiteres Python-Skript. Hierbei wird die `lgpio`-Bibliothek genutzt. Bei einer Freigabe durch das Backend steuert der Node-RED Flow das Skript den Servomotor an, um die Schranke zu öffnen. Nach einer Wartezeit von 5 Sekunden schließt sich die Schranke wieder. Nach jeder Bewegung wird das Steuersignal getrennt, um den Servomotor nicht dauerhaft mit Strom zu versorgen.

3.4 Testverfahren

Beim Testen wurden die einzelnen Komponenten schrittweise verifiziert. Zunächst wurde die Displayansteuerung getestet, anschließend der Servomotor-Winkel für die Schrankenbewegung. Schließlich wurde die gesamte Kette (Kameraerfassung → LPR → MQTT-Meldung → Backend-Entscheidung → Display/Schranken-Aktion) in Kombination getestet.

4 Evaluierung

4.1 Erkennungsgeschwindigkeit und Latenz

Die Kennzeichenerkennung benötigt ca. 10 bis 15 Sekunden pro Erkennungsvorgang. Diese Latenz lässt sich durch die Deep-Learning-Inferenz von YOLOv8n und EasyOCR erklären, die vollständig auf der Haupt-CPU des Raspberry Pi 5 ausgeführt werden muss. Ohne dedizierte GPU-Beschleunigung ist die Ausführung dieser neuronalen Netze ein Flaschenhals. Allerdings ist dies für den Prototyp akzeptabel.

4.2 Arbeitsspeicher-Erweiterung

Der Raspberry Pi 5 besitzt 1 GB Arbeitsspeicher. Da das Laden des YOLO-Modells und die Ausführung der EasyOCR-Engine den RAM im Betrieb überschritt, stürzte die Kennzeichenerkennung ab. Es wurde ein RAM-Swap auf dem Betriebssystem eingerichtet. Durch das Hinzufügen von 2 GB virtuellem Auslagerungsspeicher steigt der gesamte verfügbare Speicher auf 3 GB, somit läuft der LPR-Dienst stabil. Allerdings ist die Inferenzgeschwindigkeit durch die Nutzung von Swap-Speicher langsamer, da der Zugriff auf die SD-Karte deutlich langsamer ist als auf den physischen RAM.

5 Reflektion

5.1 Gewonnene Erkenntnisse

Das Projekt zeigte einige Probleme bei der Entwicklung von Edge-KI-Systemen auf ressourcenbeschränkter Hardware. Die Notwendigkeit von RAM-Erweiterungen unter Linux und die Optimierung der KI-Modelle mithilfe der Quantisierung waren entscheidend. Zudem ist die Wahl der OCR-Engine der wichtigste Faktor bei der Erkennungsgenauigkeit. EasyOCR erwies sich als robuster gegenüber den Gegebenheiten des Prototyps, während Tesseract OCR zwar schneller war, aber unzuverlässige Ergebnisse lieferte.

5.2 Schwachstellen des physischen Prototyps

Der physische Aufbau des Parkhaus-Modells brachte zwei wesentliche Probleme mit sich:

- Kameraabstand: Die Kamera war im ursprünglichen Aufbau sehr nah am Stellplatz positioniert. Da das YOLOv8n-Modell auf reale Pkw-Proportionen trainiert wurde, konnte es das Kennzeichen bei zu geringem Abstand nicht identifizieren. Der Abstand von der Kamera zum Kennzeichen musste vergrößert werden, damit das Modell Bounding-Boxes fand.
- Lichtverhältnisse und Kontrast: Die gedruckten Kennzeichen auf den Spielzeugautos besaßen bei normalem Licht keinen ausreichenden Kontrast für die OCR-Erkennung oder das YOLO-Modell. Dieses Problem konnte nur umgangen werden, indem das Kennzeichen auf einem Smartphone-Bildschirm mit maximaler Helligkeit simuliert wurde.

5.3 Zukünftige Verbesserungen

Im Rahmen des Projekts wurde überlegt, ein maßgeschneidertes Objekterkennungsmodell mithilfe von Plattformen wie Edge Impulse [6] zu trainieren. Damit hätte man Bilddaten der Spielzeugautos direkt im Modell-Parkhaus aufnehmen und labeln können.

Da es sich bei dem vorliegenden Aufbau jedoch um einen Prototyp handelt, wurde auf dieses Training verzichtet. In einer realen Umgebung werden echte Autos mit einheitlichen Kennzeichen verwendet. Für dieses Szenario existieren bereits fertig trainierte LPR-Modelle, die out-of-the-box funktionieren. Ein spezielles Modell-Training auf Spielzeugautos wäre somit nur für den Prototyp nützlich gewesen. Für zukünftige Tests des Prototyps wäre zudem der Einsatz eines AI-Beschleunigers (z. B. das Raspberry Pi AI Kit mit Hailo-8L NPU) der sinnvollste Ansatz, um Echtzeit zu gewährleisten.

Zudem ist das Thema Datenschutz und DSGVO-Konformität bei der Kennzeichenerkennung ein wichtiger Aspekt, der in zukünftigen Arbeiten berücksichtigt werden sollte. Eine mögliche Lösung wäre die Anonymisierung der Kennzeichen mittels Hashing oder die Implementierung

von Datenschutzrichtlinien, um die Privatsphäre der Fahrzeughalter zu schützen. Zudem ist das WLAN mit einem einfachen Passwort gesichert, was in einer realen Umgebung nicht ausreichend wäre. Hier sollte eine sichere Authentifizierung und Verschlüsselung implementiert werden.

Auch das Modelle werden auf hellen kontrastreichen Kennzeichen trainiert. In einer realen Umgebung mit verschmutzten, beschädigten oder schlecht beleuchteten Kennzeichen könnte die Erkennungsrate deutlich sinken. Hier wäre ein robusteres Modell oder Infrarot-Kamerasystem eine mögliche Verbesserung.

Literatur

- [1] Joseph Redmon u. a. *You Only Look Once: Unified, Real-Time Object Detection*. 2015. arXiv: [1506.02640](https://arxiv.org/abs/1506.02640). URL: <http://arxiv.org/abs/1506.02640>.
- [2] Rakpong Kittinaradorn und JaideAI contributors. *EasyOCR*. Version 1.7.2. 2020. URL: <https://github.com/jaivedai/easyocr>.
- [3] Tencent. *NCNN: Hocheffizientes neuronales Netzwerk-Inferenz-Framework für mobile Plattformen*. 2020. URL: <https://github.com/Tencent/ncnn>.
- [4] D.R. Vedhaviyassh u. a. *Comparative Analysis of EasyOCR and TesseractOCR for Automatic License Plate Recognition using Deep Learning Algorithm*. 2022. DOI: [10.1109/ICECA55336.2022.10009215](https://doi.org/10.1109/ICECA55336.2022.10009215).
- [5] K. Navaneet. *ModernLPR_system: End-to-End Automatic License Plate Recognition System*. GitHub-Repository für moderne, neuronale Kennzeichenerkennungssysteme. 2026. URL: https://github.com/navaneet625/ModernLPR_system.
- [6] Edge Impulse Inc. *Edge Impulse: The Leading Edge AI Platform*. <https://www.edgeimpulse.com/>. Accessed: 2026-07-18. 2026.

KI-Klausel

Gemini wurde punktuell zur sprachlichen Überarbeitung und Formulierung einzelner Textstellen sowie als Hilfsmittel bei der Erstellung des Komponentendiagramms eingesetzt. Die Konzeption und Umsetzung des Projekts sowie die fachlichen Inhalte der Dokumentation stammen vom Verfasser. Sämtliche Vorschläge wurden eigenständig geprüft und bei Bedarf angepasst.